

Epitech Technology

MSc2

Travel Order Resolver

Extraction d'itinéraires de train à partir de requêtes en langage naturel

Rapport d'expérimentation

Équipe de projet

- Adrien GARNIER
- Chi NGUYEN
- Nathan HAN
- Samir MIGUEL
- Zeyu HOU

Date : 5 mars 2026

Année académique 2025–2026

Table des matières

1	Introduction	1
1.1	Contexte	1
1.2	Objectifs du projet	1
2	Modèles de traitement du langage	2
2.1	Baseline à base de règles	2
2.2	Présentation de spaCy	2
2.2.1	Modèles sm, md, lg	2
2.2.2	Modèle spaCy <code>blank</code> (coquille vide)	3
2.3	Présentation de CamemBERT	3
2.3.1	Impact de la taille du dataset	3
2.3.2	Impact du learning rate, batch size et scheduler	3
3	Protocole expérimental	4
3.1	Dataset et splits	4
3.2	Entraînement des modèles spaCy	4
3.3	Entraînement des modèles CamemBERT	4
3.4	Post-traitement des entités	5
4	Résultats et analyses	6
4.1	Résultats comparatifs (meilleurs modèles)	6
4.2	Observations clés	6
5	Module Pathfinding	7
5.1	Algorithmes disponibles	7
5.2	Fonctionnalités SNCF	7
5.3	Réseau de gares et exemples	7
6	MVP0, interface et exécution	8
6.1	MVP0 et choix de modèle	8
6.2	Interface utilisateur minimale	8
6.3	Modes d'exécution	8
7	Conclusion	9
A	Annexe : Courbe de loss spaCy	10

Chapitre 1

Introduction

1.1 Contexte

L’objectif du projet *Travel Order Resolver* est de transformer une demande de voyage en français, formulée en langage naturel, en un itinéraire de train optimal au sein du réseau ferroviaire français. Par exemple, à partir de la phrase « Je voudrais aller de Paris à Lyon », le système doit extraire automatiquement l’origine et la destination, puis calculer l’itinéraire le plus rapide entre ces deux gares.

Le système repose sur trois briques principales :

- un module de NLP pour l’extraction d’entités nommées (Named Entity Recognition, NER) d’origine et de destination (spaCy, CamemBERT, baseline) ;
- un module de pathfinding basé sur plusieurs algorithmes (Dijkstra, A*, recherche du chemin avec le moins de correspondances) ;
- un module d’accès aux données SNCF (API, fuzzy matching des noms de villes, recherche de la gare la plus proche).

1.2 Objectifs du projet

Le projet vise à concevoir et évaluer une chaîne complète allant de la requête en langage naturel jusqu’à la proposition d’itinéraire optimal, en :

- comparant différents modèles de NER (baseline, spaCy, CamemBERT) ;
- étudiant l’impact de la taille du dataset (1k, 3k, 5k, full) et des hyperparamètres (learning rate, batch size, scheduler) ;
- évaluant plusieurs stratégies de pathfinding (Dijkstra, A*, min correspondances) ;
- intégrant des mécanismes robustes de résolution de gares (fuzzy matching, gare la plus proche).

Chapitre 2

Modèles de traitement du langage

2.1 Baseline à base de règles

La baseline est un modèle « fait maison » basé sur :

- des règles linguistiques simples (détection de prépositions de type « de », « à », « vers ») ;
- un gazetteer de gares et de villes SNCF ;
- des heuristiques pour décider quels tokens avant/après la préposition correspondent à l'origine ou à la destination.

Cette baseline atteint :

- **Accuracy** : 67,13 % ;
- **F1 Score global** : 88,41 % ;
- temps d'inférence autour de 17,38 ms par phrase.

Ce modèle montre rapidement ses limites : forte dépendance aux formulations exactes, difficulté avec les noms composés et la ponctuation, et maintenance complexe lorsque le vocabulaire évolue.

2.2 Présentation de spaCy

spaCy est une bibliothèque de NLP en Python orientée production, qui fournit des pipelines pré-entraînés pour différentes langues : tokenisation, POS tagging, parsing, NER, etc. Les pipelines français typiques incluent `fr_core_news_sm`, `fr_core_news_md` et `fr_core_news_lg`.

2.2.1 Modèles *sm*, *md*, *lg*

Les modèles *sm*, *md* et *lg* sont des pipelines généralistes :

- `ner_sm_full (fr_core_news_sm)` : 52,32 % d'accuracy, 86,93 % de F1, 11,9 ms d'inférence ;
- `ner_md_full (fr_core_news_md)` : 54,94 % d'accuracy, 90,55 % de F1, 13,1 ms ;
- `ner_lg_full (fr_core_news_lg)` : 55,03 % d'accuracy, 90,26 % de F1, 8,5 ms.

Ces modèles restent nettement en dessous des modèles *blank* spécialisés sur la tâche ORIGIN/DESTINATION.

2.2.2 Modèle spaCy blank (coquille vide)

Les modèles `blank` partent d'un pipeline vide (`spacy.blank("fr")`) dans lequel seul un composant NER est appris sur le dataset spécialisé. Les principales variantes entraînées sont :

- `ner_blank_1000` (989 samples) : 67,72 % d'accuracy, 94,92 % de F1, 5,3 ms ;
- `ner_blank_3000` (2965 samples) : 70,00 % d'accuracy, 95,86 % de F1, 5,2 ms ;
- `ner_blank_5000` (4945 samples) : 69,38 % d'accuracy, 95,98 % de F1, 9,5 ms ;
- `ner_blank_full` (8352 samples) : 69,29 % d'accuracy, 95,92 % de F1, 5,4 ms.

Les modèles *blank* surpassent nettement les modèles pré-entraînés `sm/md/lg` en accuracy (67–70 % contre 52–55 %) et en F1 (96 % contre 87–91 %), ce qui montre que les entités NER génériques pré-entraînées peuvent interférer avec l'apprentissage de `ORIGIN/DESTINATION`.

2.3 Présentation de CamemBERT

CamemBERT est un modèle de langue français basé sur l'architecture RoBERTa, lui-même dérivé de BERT (Bidirectional Encoder Representations from Transformers). Il est pré-entraîné sur un large corpus français (données OSCAR, etc.) via une tâche de *Masked Language Modeling* et obtient d'excellentes performances sur diverses tâches aval.

2.3.1 Impact de la taille du dataset

Les variantes principales sont :

- `camembert_ner_1000` (1000 samples) : 71,48 % d'accuracy, 96,53 % de F1, 161 ms ;
- `camembert_ner_3000` (3000 samples) : **75,42 % d'accuracy, 98,03 % de F1**, 108 ms ;
- `camembert_ner_5000` (5000 samples) : 72,88 % d'accuracy, 97,34 % de F1, 112 ms ;
- `camembert_ner_full` (8462 samples) : 72,00 % d'accuracy, 97,06 % de F1, 96 ms.

Le modèle `camembert_ner_3000` apparaît comme le meilleur compromis (75,42 % d'accuracy, 98,03 % de F1, 108 ms).

2.3.2 Impact du learning rate, batch size et scheduler

Sur le dataset complet, plusieurs expériences hyperparamètres ont été menées :

- **Learning rate** :
 - `camembert_lr1e-05` : 75,07 % d'accuracy, 98,00 % de F1, 78 ms ;
 - `camembert_lr3e-05` : 72,18 % d'accuracy, 96,88 % de F1, 91 ms ;
 - `camembert_lr5e-05` : 71,92 % d'accuracy, 97,03 % de F1, 80 ms.
- **Batch size** :
 - `camembert_bs32` : 72,35 % d'accuracy, 97,14 % de F1, 80 ms ;
 - `camembert_bs16` : 71,83 % d'accuracy, 96,91 % de F1, 81 ms ;
 - `camembert_bs8` : 70,87 % d'accuracy, 96,65 % de F1, 78 ms.
- **Scheduler** :
 - `camembert_sched_cosine` : 71,74 % d'accuracy, 96,88 % de F1, 134 ms ;
 - `camembert_sched_linear` : 71,13 % d'accuracy, 96,65 % de F1, 135 ms.

Un learning rate plus faible (1e-5) améliore légèrement les performances globales, avec une F1 de 98,00 % et un temps d'inférence réduit à 78 ms.

Chapitre 3

Protocole expérimental

3.1 Dataset et splits

Le dataset complet est constitué d'environ 10 000 phrases, générées à partir de templates et de listes de villes/gares SNCF. Il est découpé ainsi :

- *train* : 7000 exemples ;
- *validation* : 1500 exemples ;
- *test* : 1500 exemples.

Les expérimentations principales ont été menées sur des sous-ensembles de 1000, 3000, 5000 et sur le dataset complet (*full*).

3.2 Entraînement des modèles spaCy

Les modèles spaCy sont entraînés via `train_spacy_models.py` avec :

- option `-all-experiments` pour lancer `blank_1k`, `blank_3k`, `blank_5k`, `blank_full`, `sm_full`, `md_full`, `lg_full` ;
- `-size` pour sélectionner la taille (1000, 3000, 5000 ou `full`) ;
- `-base` pour choisir la base (`blank`, `sm`, `md`, `lg`) ;
- `-n-iter` pour le nombre d'itérations d'entraînement (par défaut 30).

Les métriques détaillées sont exportées dans `outputs/evaluation/spacy_experiments.json` et les courbes de loss dans `spacy_loss_curves.png`.

3.3 Entraînement des modèles CamemBERT

Les modèles CamemBERT sont entraînés via `train_camembert.py` avec :

- `-all-experiments` pour entraîner les tailles 1k, 3k, 5k, `full` et les variations de LR/batch/scheduler ;
- `-train-size` pour sélectionner le nombre de samples ;
- `-epochs` (par défaut 3) ;
- `-batch-size` (par défaut 16) ;
- `-learning-rate` (par défaut 5e-5) ;
- `-lr-scheduler` (`linear` ou `cosine`).

Les résultats sont enregistrés dans `outputs/evaluation/camembert_experiments.json`, et les modèles dans `models/camembert_ner/`.

3.4 Post-traitement des entités

Comme dans les versions précédentes, une fonction `_clean_entity()` est utilisée pour nettoyer les entités (trim des espaces, suppression de la ponctuation finale), ce qui améliore sensiblement les métriques d'accuracy en évitant des mismatches sur des détails de format.

Chapitre 4

Résultats et analyses

4.1 Résultats comparatifs (meilleurs modèles)

Modèle	Accuracy	F1 Score	Origin F1	Dest F1	Inférence
CamemBERT 3k	75,42 %	98,03 %	97,89 %	98,17 %	108 ms
CamemBERT lr=1e-5	75,07 %	98,00 %	–	–	78 ms
spaCy blank 5k	69,38 %	95,98 %	96,27 %	95,68 %	9,5 ms
spaCy blank 3k	70,00 %	95,86 %	96,33 %	95,38 %	5,2 ms
Baseline	67,13 %	88,41 %	–	–	17,38 ms

Les deux modèles recommandés sont :

- **CamemBERT 3k** pour la meilleure précision globale (98,03 % de F1, 75,42 % d'accuracy, 108 ms) ;
- **spaCy blank 3k** pour la rapidité (95,86 % de F1, 70,00 % d'accuracy, 5,2 ms, soit environ 20 fois plus rapide).

4.2 Observations clés

- **3000 samples est le sweet spot** : pour spaCy et CamemBERT, 3000 samples donnent les meilleurs résultats globaux ; au-delà, les gains sont marginaux voire légèrement négatifs.
- **Les modèles blank spaCy battent les pré-entraînés** : les modèles sm/md/lg (52–55 % d'accuracy) sous-performent largement les modèles blank (67–70 %), car les entités NER génériques interfèrent avec ORIGIN/DESTINATION.
- **Un LR plus faible améliore CamemBERT** : un learning rate de 1e-5 (98,00 % de F1) surpasse 5e-5 (97,03 % de F1) sur le dataset complet.
- **Le post-processing est crucial** : la méthode `_clean_entity()` supprime les espaces et la ponctuation parasites, améliorant fortement l'accuracy.

Chapitre 5

Module Pathfinding

5.1 Algorithmes disponibles

Le module `src/pathfinder/dijkstra.py` utilise NetworkX pour calculer différents types d'itinéraires :

Algorithme	Méthode	Description
Dijkstra	<code>find_shortest_path()</code>	Plus court chemin par durée
A*	<code>find_path_astar()</code>	Heuristique Haversine / vitesse TGV
Min-changes	<code>find_path_min_changes()</code>	Minimise les correspondances
Alternatives	<code>find_all_paths()</code>	K plus courts chemins

5.2 Fonctionnalités SNCF

Le client `SNCFClient` fournit plusieurs fonctionnalités avancées :

- **Fuzzy matching** des noms de villes (distance de Levenshtein, seuil 0,8) ;
- recherche de la **gare la plus proche** via distance Haversine lorsque la ville ne possède pas de gare ;
- vérification géographique pour détecter les villes hors France métropolitaine.

5.3 Réseau de gares et exemples

Le graphe ferroviaire comporte 51 gares principales et 142 connexions (TGV, Intercités, TER). Quelques exemples de trajets :

Trajet	Durée	Distance	Correspondances
Paris → Marseille	2h48	666 km	0 (TGV)
Lyon → Bordeaux	3h22	657 km	2
Lille → Nice	5h30	1100 km	1

Chapitre 6

MVP0, interface et exécution

6.1 MVP0 et choix de modèle

Le MVP0 doit :

- accepter une phrase en français ;
- extraire correctement l'origine et la destination ;
- fournir un itinéraire cohérent (durée, distance, correspondances) ;
- répondre dans un temps acceptable.

Deux configurations sont privilégiées :

- **Production haute précision** : CamemBERT 3k ou CamemBERT full lr=1e-5 ;
- **Production haute volumétrie** : spaCy blank 3k (ou 5k) pour une latence minimale.

6.2 Interface utilisateur minimale

Une interface simple suffit :

- un champ de texte pour saisir « Je veux aller de Paris à Marseille » ;
- un bouton « Résoudre » ;
- un affichage de la route (segments, durée, distance, correspondances).

6.3 Modes d'exécution

Quelques commandes typiques :

- **Baseline** :

```
python scripts/travel_resolver.py -m baseline "Je veux aller de Paris à Lyon"
python scripts/travel_resolver.py -m baseline --interactive
```

- **spaCy** :

```
python scripts/travel_resolver.py "Je veux aller de Paris à Marseille"
python scripts/travel_resolver.py --interactive
```

- **CamemBERT** :

```
python scripts/travel_resolver.py -m camembert "Je veux aller de Paris à Lyon"
python scripts/travel_resolver.py -m camembert --interactive
```

Chapitre 7

Conclusion

Les expérimentations montrent qu’une combinaison de NER spécialisé (spaCy blank ou CamemBERT) et d’algorithmes de plus court chemin (Dijkstra, A*) permet de transformer de manière robuste des requêtes en langage naturel en itinéraires de train optimaux, tout en offrant des marges d’optimisation importantes via les hyperparamètres et les choix d’architecture.

Annexe A

Annexe : Courbe de loss spaCy

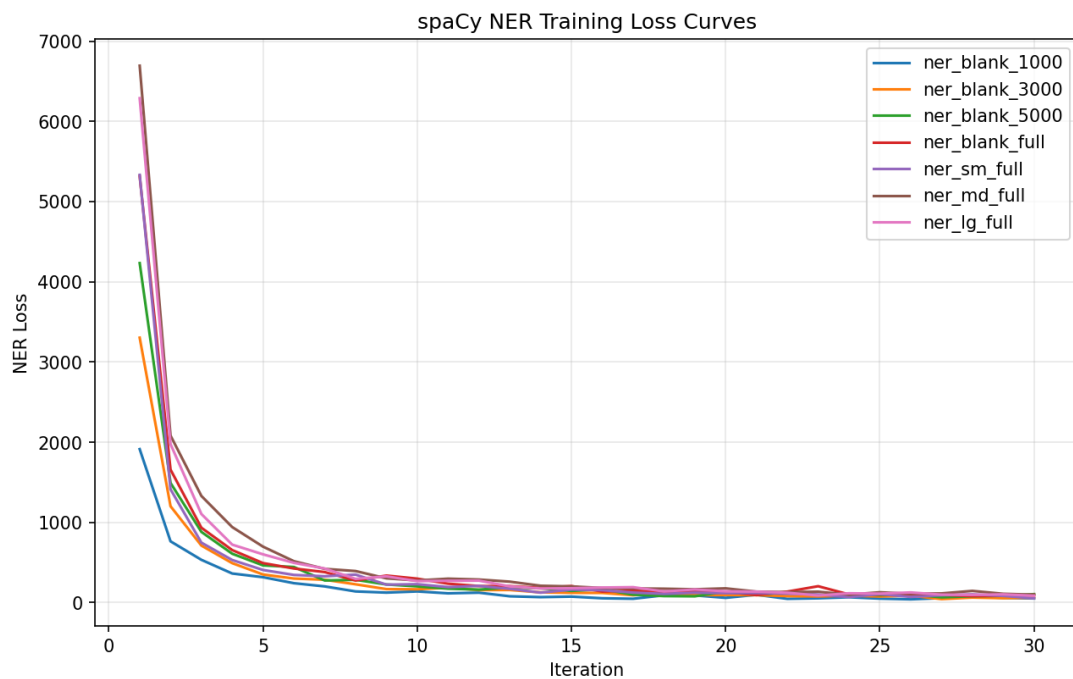


FIGURE A.1 – Courbe de loss d'entraînement des modèles spaCy (spacy_training_loss.png).